



Fullstack Web Development

Session 3 | Week 3/Day 1 | 120 min

Tailwind CSS

Instructor: Ms. Liev Nova



090667393



What Is Tailwind CSS ?



What is Tailwind CSS?

Tailwind CSS is a **utility-first CSS framework** for building modern user interfaces quickly.

Instead of writing custom CSS, developers use **small utility classes directly in HTML**.

Example: Traditional CSS

```
.button {  
  background-color: blue;  
  color: white;  
  padding: 10px;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

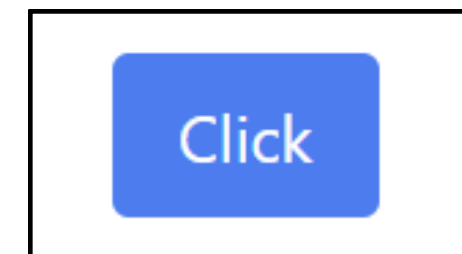
```
<button class="button">Click</button>
```



Example: Tailwind CSS

```
<button class="bg-blue-500 text-white px-4 py-2 rounded">  
  Click  
</button>
```

Design directly in HTML using utility classes



Utility-First Concept

What is Utility-First CSS?

A **utility-first framework** provides **small, single-purpose classes** that control one specific style.

Instead of writing custom CSS classes, you combine **many utility classes directly in HTML**.

Each class performs **one styling job**.

Example utilities:

- text-center
- bg-blue-500
- p-4
- rounded



These small classes combine together to create a complete design.

Traditional CSS Approach

In traditional CSS development, the styling process usually follows this workflow:

HTML → class name → CSS file → styling

Example:

```
.button {  
  background-color: blue;  
  color: white;  
  padding: 10px;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
}
```



```
<button class="button">Click</button>
```

1. Create a class name
2. Write custom CSS rules
3. The HTML references the CSS class
4. The CSS file controls the styling

Problems With This Approach

- Too many CSS files
- Hard to maintain large projects
- Class naming becomes difficult
- Unused CSS increases file size
- Switching between HTML and CSS constantly

Tailwind CSS Approach

Tailwind changes the workflow.

Instead of writing CSS, you use **pre-built utility classes** directly in HTML.

HTML → Utility Classes → Styling

Example:

```
<button class="bg-blue-500 text-white px-4 py-2 rounded">  
  Click  
</button>
```

*No CSS file needed.
All styles come from **Tailwind utility classes**.*

Tailwind CSS Approach

How Utility Classes Work

Each Tailwind class does **only one styling task**.

Class	Purpose	CSS Equivalent
bg-blue-500	Sets background color	background-color
text-white	Sets text color	color
px-4	Horizontal padding	padding-left & padding-right
py-2	Vertical padding	padding-top & padding-bottom
rounded	Adds rounded corners	border-radius

Installing Tailwind CSS v4

Installation Options

- CDN
- CLI
- Framework integration
 - React
 - Vue
 - Next.js
 - Laravel

Installing Tailwind CSS v4

Installing Tailwind CSS (CDN Method)

What is CDN Installation?

CDN (Content Delivery Network) allows us to **use Tailwind CSS instantly without installing anything.**

We simply add a **script link inside the HTML file.**

Installing Tailwind CSS v4

Installing Tailwind CSS (CDN Method)

Add Tailwind CDN Script

Add this line inside the `<head>` section of your HTML file.

```
<script src="https://cdn.tailwindcss.com"></script>
```

This method is **great for learning and quick prototypes**.

***Note:** CDN is **not recommended for production projects**.

Installing Tailwind CSS v4

Installing Tailwind CSS (CDN Method)

Example HTML Page Using Tailwind

```
<!DOCTYPE html>
<html lang="en">

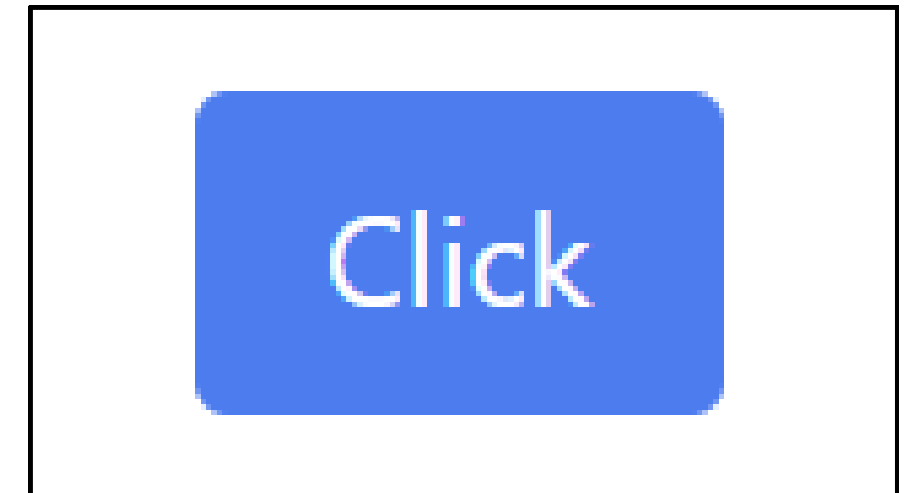
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Tailwind CSS Button</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="p-6">
  <button class="bg-blue-500 text-white px-4 py-2 rounded">
    Click
  </button>
</body>

</html>
```



Result



Installing Tailwind CSS v4

Installing Tailwind CSS Using NPM (Recommended for Projects)

Requirements:

- **Node.js** installed

```
C:\Users\lievnova>node --version  
v22.19.0
```

[01]

Install Tailwind CSS

Install `'tailwindcss'` and `'@tailwindcss/cli'` via npm.

Terminal

```
npm install tailwindcss @tailwindcss/cli
```

Installing Tailwind CSS v4

Installing Tailwind CSS Using NPM (Recommended for Projects)

Requirements:

- **Node.js** installed

```
C:\Users\lievnova>node --version  
v22.19.0
```

[02]

Import Tailwind in your CSS

Add the `@import "tailwindcss";` import to your main CSS file.

src/input.css

```
@import "tailwindcss";
```


Installing Tailwind CSS v4

Installing Tailwind CSS Using NPM (Recommended for Projects)

Requirements:

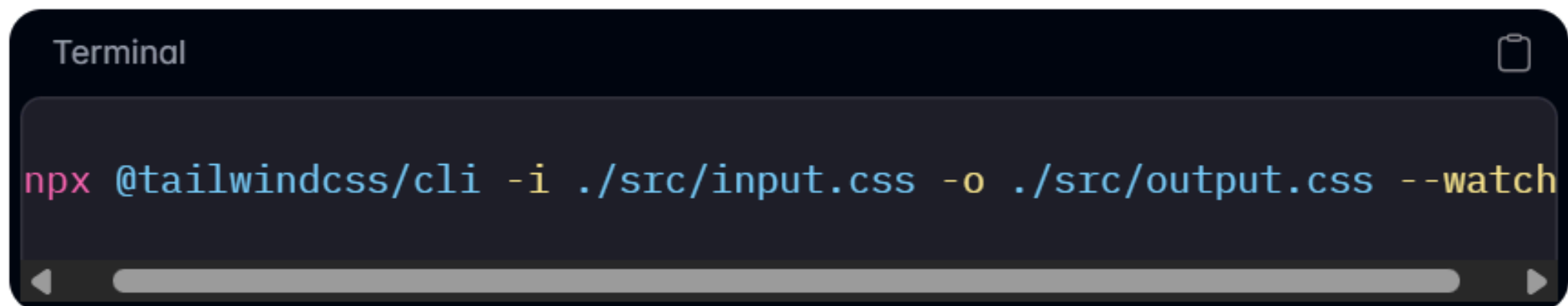
- **Node.js** installed

```
C:\Users\lievnova>node --version  
v22.19.0
```

[03]

Start the Tailwind CLI build process

Run the CLI tool to scan your source files for classes and build your CSS.

A terminal window with a dark background and light text. The title bar says "Terminal". The command entered is `npx @tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch`. The command is highlighted in a light blue color. There is a scrollbar at the bottom of the terminal window.

```
Terminal  
npx @tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch
```

Installing Tailwind CSS v4

Installing Tailwind CSS Using NPM (Recommended for Projects)

Requirements:

- **Node.js** installed

```
C:\Users\lievnova>node --version  
v22.19.0
```

[04]

Start using Tailwind in your HTML

Add your compiled CSS file to the ``<head>`` and start using Tailwind's utility classes to style your content.



src/index.html

```
<!doctype html>  
<html>  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-s<br>  
  <link href="./output.css" rel="stylesheet">  
</head>  
<body>  
  <h1 class="text-3xl font-bold underline">  
    Hello world!  
  </h1>  
</body>  
</html>
```

Installing Tailwind CSS v4

Installing Tailwind CSS Using NPM (Recommended for Projects)

Requirements:

- **Node.js** installed

```
C:\Users\lievnova>node --version  
v22.19.0
```

5. Setup Script for run build CSS in *package.json*

```
{  
  "scripts": {  
    "build:css": "npx @tailwindcss/cli -i ./src/input.css -o ./src/output.css --watch"  
  },  
  "dependencies": {  
    "@tailwindcss/cli": "^4.2.1",  
    "tailwindcss": "^4.2.1"  
  }  
}
```

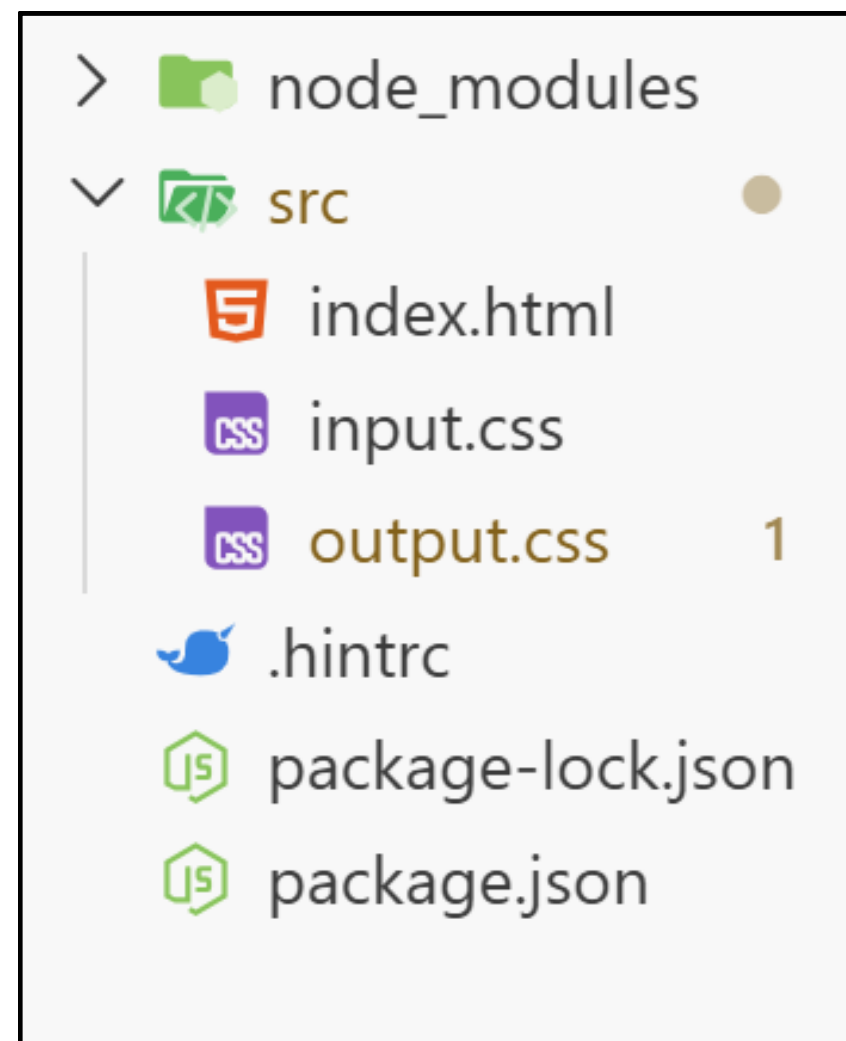
```
npm run build:css
```

6.Run Script

Installing Tailwind CSS v4

Tailwind Project Structure

Example:



**Note: In styles.css*

```
@import "tailwindcss";
```

Utility Classes

Color System

Tailwind uses color scale

Example

- blue-50
- blue-100
- blue-300
- blue-500
- blue-700
- blue-900

Example

```
<p class="text-3xl text-red-500 underline">Hello</p>
```



Hello

Utility Classes

Tailwind Spacing System

Tailwind uses a **spacing scale system**.

Spacing values are based on **rem units**.

Class	Size	CSS Value
0	0px	0
1	4px	0.25rem
2	8px	0.5rem
3	12px	0.75rem
4	16px	1rem
5	20px	1.25rem
6	24px	1.5rem
8	32px	2rem
10	40px	2.5rem
12	48px	3rem
16	64px	4rem

Utility Classes

Spacing Utilities

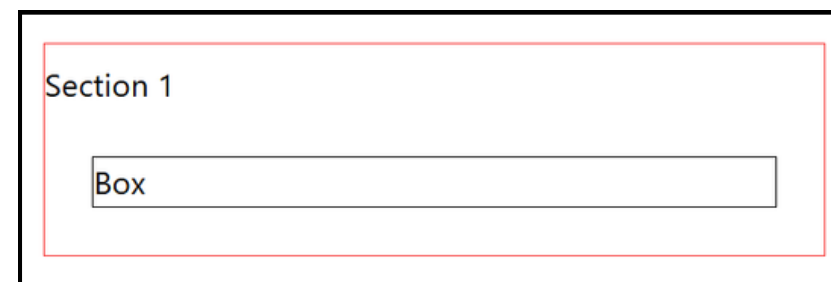
Spacing utilities control the **space inside and outside elements**.

There are **two main** types:

Type	Purpose
Padding	Space inside the element
Margin	Space outside the element

Example

```
<div class="border border-red-500">  
  <section class="mt-2">  
    <p>Section 1</p>  
  </section>  
  <section class="p-6">  
    <p class="border">Box</p>  
  </section>  
</div>
```



Utility Classes

Padding Utilities

Padding controls **space inside the element**.

Structure: `p-{size}`

Padding Direction Utilities

Padding can also be applied in **specific directions**.

Class	Meaning
p-4	padding all sides
px-4	horizontal padding
py-4	vertical padding
pt-4	padding-top
pb-4	padding-bottom
pl-4	padding-left
pr-4	padding-right

Utility Classes

Margin Utilities

Margin controls **space outside the element**.

Structure: `m-{size}`

Example

```
<div class="mt-6 mb-2 bg-red-200">  
  Example Box  
</div>
```



Example Box

Margin Direction Utilities

Margin can also be applied in **specific directions**.

Class	Meaning
m-4	margin all sides
mx-4	horizontal margin
my-4	vertical margin
mt-4	margin-top
mb-4	margin-bottom
ml-4	margin-left
mr-4	margin-right

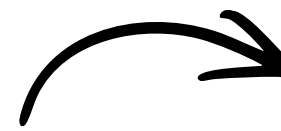
Utility Classes

Margin Utilities

- **Auto Margin** : Tailwind supports **auto margins** for layout control.

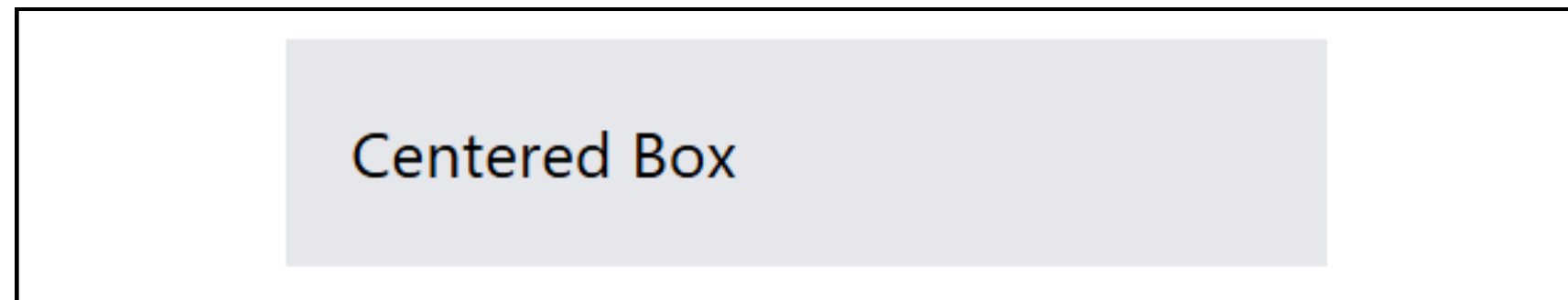
Example

```
<div class="w-64 mx-auto bg-gray-200 p-4">  
  Centered Box  
</div>
```



Class	Meaning
mx-auto	margin-left: auto
	margin-right: auto

This is commonly used to center elements horizontally.



Utility Classes

Space Between Elements

Tailwind provides utilities to add **space between child elements automatically**.

Structure:

- `space-x-{size}`
- `space-y-{size}`

Utility	Meaning
<code>space-x</code>	horizontal space between children
<code>space-y</code>	vertical space between children

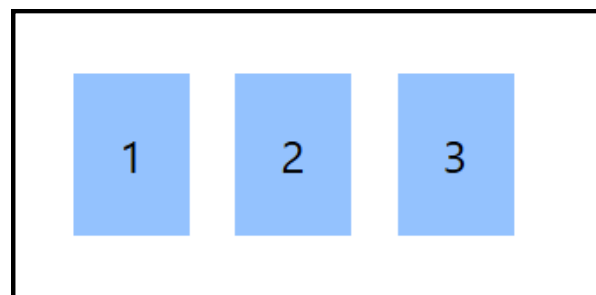
Utility Classes

Space Between Elements

Horizontal Spacing (space-x)

Example:

```
<div class="flex space-x-4">  
  <div class="bg-blue-300 p-4">1</div>  
  <div class="bg-blue-300 p-4">2</div>  
  <div class="bg-blue-300 p-4">3</div>  
</div>
```

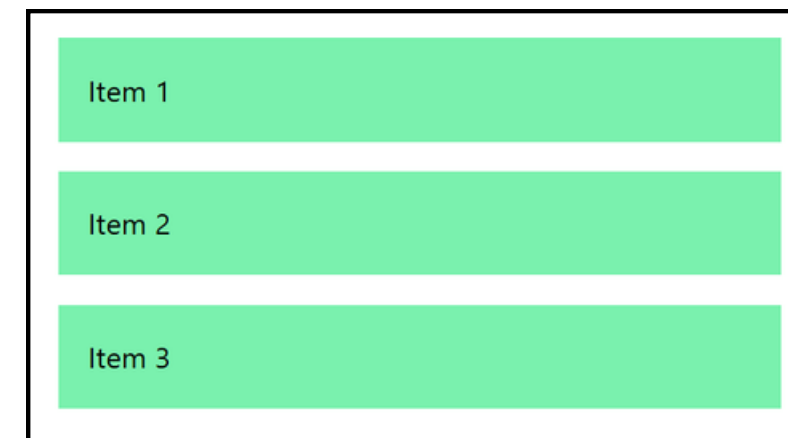


Space Between Elements

Vertical Spacing (space-y)

Example:

```
<div class="space-y-4">  
  <div class="bg-green-300 p-4">Item 1</div>  
  <div class="bg-green-300 p-4">Item 2</div>  
  <div class="bg-green-300 p-4">Item 3</div>  
</div>
```



Utility Classes

Negative Spacing

Tailwind supports negative margin values.

Example:

```
<div class="-mt-4">  
  Example  
</div>
```

This is useful for:

- *overlapping layouts*
- *advanced UI positioning*

Utility Classes

Flexbox Layout

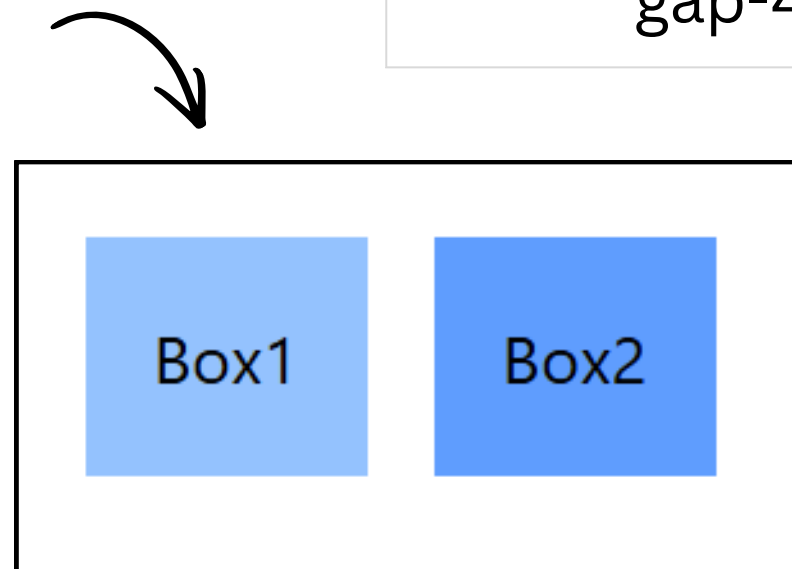
Tailwind makes layout very easy.

Example:

```
<section>
  <div class="flex gap-4">
    <div class="bg-blue-300 p-4">Box1</div>
    <div class="bg-blue-400 p-4">Box2</div>
  </div>
</section>
```

Common classes:

Class	Meaning
flex	enable flex
gap-4	space between



Utility Classes

Flexbox Layout

Flex Direction

Control direction of items:

Class	Meaning
flex-row	horizontal (default)
flex-row-reverse	reverse row
flex-col	vertical
flex-col-reverse	reverse column

Justify Content (Horizontal Alignment)

Controls alignment on main axis.

Class	Meaning
justify-start	left
justify-center	center
justify-end	right
justify-between	space between
justify-around	space around
justify-evenly	equal spacing

Utility Classes

Flexbox Layout

Align Items (Vertical Alignment)

Control direction of items:

Class	Meaning
items-start	top
items-center	center
items-end	bottom
items-stretch	stretch
items-baseline	align text

Flex Wrap

Controls whether items wrap to next line.

Class	Meaning
flex-wrap	allow wrap
flex-nowrap	no wrap
flex-wrap-reverse	reverse wrap

Utility Classes

Typography

Control text styles easily.

Example:

```
<section>
  <h1 class="text-4xl font-bold">
    Title
  </h1>
  <p class="text-gray-600 text-lg">
    Paragraph
  </p>
</section>
```



Title
Paragraph

Common classes:

Class	Meaning
text-sm	small
text-lg	large
font-bold	bold
text-center	center

Utility Classes

Responsive Design

Tailwind uses mobile-first breakpoints.

Example:

```
<section>
  <div class="text-sm md:text-lg lg:text-2xl">
    Responsive Text
  </div>
</section>
```

Responsive Text

Responsive Text

Breakpoints:

Prefix	Screen
sm	≥640px
md	≥768px
lg	≥1024px
xl	≥1280px

Dark Mode

Tailwind enable dark mode easily.

Example:

```
<div class="bg-white dark:bg-gray-900 text-black dark:text-white">  
  Dark Mode Example  
</div>
```


Tailwind v4 Customization (@theme)

New in Tailwind v4: CSS-first configuration

Instead of config file, we use:

```
@theme {  
  --color-primary: #3b82f6;  
}
```

What is **@theme**?

It defines design tokens like:

- colors
- spacing
- fonts

Using Custom Theme

```
<div class="bg-[--color-primary] text-white p-4">  
  Primary Box  
</div>
```

Tailwind Layers

What are Tailwind Layers?

Tailwind organizes styles into **three main layers**:

- **Base**
- **Components**
- **Utilities**

These layers control **how styles are applied and overridden**.

Layer Order: Base → Components → Utilities

Tailwind Layers

@layer base — Default / Global Styles

The **base layer** is used for:

- Reset styles (like normalize.css)
- Default HTML styling
- Global typography

Example:

```
@layer base {  
  h1 {  
    font-size: 2rem;  
    font-weight: bold;  
  }  
  
  body {  
    font-family: sans-serif;  
  }  
}
```

Tailwind Layers

@layer components – UI Components

The **components layer** is used to create:

- Buttons
- Cards
- Forms
- UI blocks

Example:

```
@layer components {  
  .btn-primary {  
    @apply px-4 py-2 bg-blue-500 text-white rounded;  
  }  
}
```



```
<button class="btn-primary">  
  Save  
</button>
```

Tailwind Layers

@layer utilities — Custom Utility Classes

The **utilities layer** is for:

- Small helper classes
- One-purpose styles

Example:

```
@layer utilities {  
  .text-shadow {  
    text-shadow: 2px 2px 5px gray;  
  }  
}
```




THANK YOU

For your attention !

